Basin MCMC: Discussion

Stella Srzich (3371930)

July 31, 2026

This document is meant to serve as a discussion of the Basin MCMC algorithm.

1 What kind of algorithm do we want?

TODO. In general, we want an algorithm that correctly samples from a target density $\pi(\cdot)$ (i.e., produces a Markov chain that is ergodic with respect to $\pi(\cdot)$). Moreover, we would like the chain to converge to the target density within a reasonable number of iterations. Moreover, out of all such algorithms, we would like to minimize the computational cost per effective sample.

The standard algorithm that indeed correctly samples from a target density $\pi(\cdot)$ is the Metropolis–Hastings (MH) algorithm.

Algorithm 1 Metropolis–Hastings (MH)

function: $[x^1, \ldots, x^N] = \mathrm{MH}\big(\pi(\cdot), q(\cdot \mid \cdot), x^0, N\big)$
input: target density $\pi(\cdot)$, proposal density $q(\cdot \mid \cdot)$, initial state x^0, number of steps N
output: ordered list of states $[x^1, \ldots, x^N]$

1: **for** $j = 0$ to $N - 1$ **do**
2: Given x^j, generate a proposal y distributed as $q(y \mid x^j)$.
3: Accept proposal y as the next state, i.e. set $x^{j+1} = y$, with probability

$$\alpha(y \mid x^j) = \min\left\{1, \frac{\pi(y)q(x^j \mid y)}{\pi(x^j)q(y \mid x^j)}\right\}.$$

4: Otherwise reject y and set $x^{j+1} = x^j$.
5: **end for**

However, when the target density $\pi(\cdot)$ is multimodal (e.g., as is the case when we use high-level representations for states), with basic choices of proposal kernel $q(\cdot \mid \cdot)$, the MH algorithm can struggle to explore all modes within a reasonable number of iterations. In this case, the MH algorithm will not converge to the target density within a reasonable number of iterations.

One way to address this is to, at each iteration, run a subchain of length n that targets a coarse (i.e., cheap to evaluate) density $\pi_C(\cdot)$, and then use the resulting state as a proposal for this iteration. The idea is that if the coarse density $\pi_C(\cdot)$ approximates the target density $\pi(\cdot)$ well enough, after a sufficient number of subchain steps, the proposal will be in a different mode. This effectively produces a new proposal kernel (from the basic proposal kernel $q(\cdot \mid \cdot)$) for use in in the MH algorithm that "wormholes" between modes without us having to be clever about explicitly designing such a proposal. Or, as we like to think of it, a proposal kernel of little "rats and

mice" that tunnel between the modes. This idea is implemented in the Randomised-Length-Subchain Surrogate Transition (RST) algorithm.

Algorithm 2 Randomised-Length-Subchain Surrogate Transition (RST)

function: $[x^1, \ldots, x^N] = \mathrm{RST}\big(\pi_F(\cdot), \pi_C(\cdot), q(\cdot \mid \cdot), p(\cdot), x^0, N\big)$

input: target (fine) density $\pi_F(\cdot)$, surrogate (coarse) density $\pi_C(\cdot)$, proposal kernel $q(\cdot \mid \cdot)$, probability mass function $p(\cdot)$ over subchain length, initial state x^0, number of steps N

output: ordered list of states $[x^1, \ldots, x^N]$ (or just the final state x^N)

1: **for** $j = 0$ to $N - 1$ **do**

2: Draw the subchain length $n \sim p(\cdot)$.

3: Starting at x^j, generate a subchain of length n using MH Alg. 1 to target $\pi_C(\cdot)$:

$$y = \mathrm{MH}\big(\pi_C(\cdot), q(\cdot \mid \cdot), x^j, n\big).$$

4: Accept the proposal y as the next sample, i.e. set $x^{j+1} = y$, with probability

$$\alpha(y \mid x^j) = \min\left\{1, \frac{\pi_F(y)\pi_C(x^j)}{\pi_F(x^j)\pi_C(y)}\right\}.$$

5: Otherwise reject y and set $x^{j+1} = x^j$.

6: **end for**

To make the RST algorithm more efficient, one would like to estimate the mean and covariance of the discrepancy between the exact and coarse forward maps and subsequently modify the coarse likelihood so that the resulting coarse density more closely approximates the target density as the chain progresses (see Fox, Cui and Neumayer 2020).

If we allow state-dependent coarse densities $\pi_{C,x}(\cdot)$, as opposed to a single state-independent coarse density $\pi_C(\cdot)$ as in the RST algorithm, the coarse forward map can be locally corrected so that it agrees with the exact forward map at the current state x. For this locally corrected map, the discrepancy is zero at x and has mean zero under the stationary transition law. For some reason (I think because we no longer need to correct the global coarse model error, we only need to correct the change in the coarse model error over one move), the modified state-dependent coarse densities more closely approximate the target density than the modified state-independent coarse density. Long story short, this results in a more efficient algorithm.

We name this algorithm the Rat-Mice (RM) algorithm. For simplicity, we begin by only considering a fixed subchain length n.

Algorithm 3 Rat-Mice (RM)

function: $[x^1, \ldots, x^N] = \mathrm{RM}\big(\pi(\cdot), \{\pi_{C,x}(\cdot)\}_{x\in\mathcal{X}}, q(\cdot \mid \cdot), n, x^0, N\big)$
input: target density $\pi(\cdot)$, state-dependent surrogate (coarse) densities $\{\pi_{C,x}(\cdot)\}_{x\in\mathcal{X}}$, proposal kernel $q(\cdot \mid \cdot)$, subchain length n, initial state x^0, number of steps N
output: ordered list of states $[x^1, \ldots, x^N]$

1: **for** $j = 0$ to $N - 1$ **do**
2: Starting at x^j, generate subchain of length n using MH Alg. 1 to target $\pi_{C,x_j}(\cdot)$:

$$y = \mathrm{MH}\big(\pi_{C,x_j}(\cdot), q(\cdot \mid \cdot), x^j, n\big).$$

3: Accept the state y as our proposal, i.e. set $x^{j+1} = y$, with probability

$$\alpha(y \mid x^j) = \text{(some value that ensures the chain is in detailed balance with } \pi(\cdot)).$$

4: Otherwise reject y and set $x^{j+1} = x^j$.
5: **end for**

We now recap the point of the RM algorithm. If you use basic proposal kernels (i.e., ones you are not super clever about), the MH algorithm will not converge to a multimodal target density within a reasonable number of iterations. One way to address this is to use the basic proposal kernel to generate a more sophisticated proposal kernel that acts like little "rats and mice" that tunnel between the modes.

To generate this more sophisticated proposal kernel, you want to target a density that is cheap to evaluate (so that doing a bunch of steps in the subchain is cheap), and you want the density to be a good approximation of the target density (so that proposals kind of look like samples from the target density and are accepted with high probability). The RST algorithm does this, but only allows for a single state-independent coarse density.

Given a state-independent coarse density (which will be defined by a coarse forward map), we can easily define state-dependent coarse densities that are better approximations of the target density (and are as cheap to evaluate as they use the same coarse forward map). Thus, we want to somehow modify the RST algorithm to allow for state-dependent coarse densities.

2 The Basin MCMC algorithm

2.1 Basins

We first discuss the notion of a basin. One way to think of a basin is as a probability distribution over states that captures how *similar* states are to some representative state. In particular, states are considered to be similar to the representative state (e.g., they have high probability density within its basin) if they produce similar data (say, if we treat the data generated by the representative state as true data, then the posterior probability density of the state given the data is high).

To be more precise, a basin B is a set of the following:

- A **representative state** x_0.

- A **probability distribution** $\pi_B(\cdot)$ over states. We can think of this as consisting of:
  - A **sequence of probability distributions** $\{\pi_{B,n}(\cdot)\}_{n\in\mathbb{N}}$ each over the set of states with n atoms $\mathcal{X}_n$. Note that in cases where the order of states does not matter (as it does not in our GPR application), each $\pi_{B,n}(\cdot)$ can be entirely characterised as a distribution over each component of an atom.
  - A **sequence of probabilities** $\{p_{B,n}\}_{n\in\mathbb{N}}$ each defining the probability of a state having n atoms, given the state is in basin B (e.g., it is distributed by $\pi_B(\cdot)$).
- A **probability** p_B that the state is in basin B (e.g., it is distributed by $\pi_B(\cdot)$).
- (Maybe?) a **local sampler** that generates samples from the posterior, proposed by $\pi_B(\cdot)$, and adapts $\pi_B(\cdot)$ and p_B with each iteration.

2.2 Moves on basins

Recall that our original MCMC algorithm worked by doing a mixture of moves between states of the model, such as birth, death, and perturbation moves. The basin MCMC algorithm will work in a similar way, still doing some kind of mixture of moves, but instead we will consider the state of the MCMC to be the set of basins (which implies a state of the model by unioning the representative states of the basins), and we will do moves on these basins.

The moves we will do on basins are:

- **Birth move:** add a new basin to the state.
- **Death move:** remove a basin from the state.
- **Perturb move:** run the local sampler within the basin, adapting the probability distribution and probability of the basin, and update the representative state of the basin to be the final state of the local sampler.
- **Merge move:** merge two basins into one, where the representative state of the new basin is the union of the representative states of the two basins.
- **Split move:** split a basin into two, where the representative state of the new basins is a partition of the representative state of the original basin.

To ensure the mixture of moves is in detailed balance with the target density it suffices to ensure each of the moves is in detailed balance with the target density. To achieve this, we must combine the birth and death moves, and the merge and split moves, into a single move. We then ensure each of the birth/death, perturb, and merge/split moves is in detailed balance with the target density with an MH step.

2.3 Algorithm

The algorithm roughly works as follows:

Algorithm 4 Basin MCMC

function: $[x^1, \ldots, x^N] = \text{BasinMCMC}\big(\pi(\cdot), \{\pi_{C,x}(\cdot)\}_{x \in \mathcal{X}}, q(\cdot \mid \cdot), n, x^0, N\big)$

input: target density $\pi(\cdot)$, birth/death proposal kernel $q_{Birth/Death}(\cdot \mid \cdot)$, perturb proposal kernel $q_{Perturb}(\cdot \mid \cdot)$, merge/split proposal kernel $q_{Merge/Split}(\cdot \mid \cdot)$, initial state x^0, initial basin probability distribution $\pi_B(\cdot)$ (dependent on representative state x^0), number of steps N

output: ordered list of states $[x^1, \ldots, x^N]$

Initialize list of basins $\mathcal{B}^0 = [B]$ with $B = (x^0, \pi_B(\cdot), p_B = 1)$.
for $j = 0$ to $N - 1$ **do**
    Given $\mathcal{B}^j$, choose a move from $\{Birth/Death, Perturb, Merge/Split\}$.
    if Birth/Death: **then**
        Propose a list of basins $\mathcal{B}^*$ from $q_{Birth/Death}(\cdot \mid \mathcal{B}^j)$, where $\mathcal{B}^*$ is $\mathcal{B}^j$ with an additional or removed basin.
        Compute MH acceptance probability $\alpha(\mathcal{B}^* \mid \mathcal{B}^j)$ for Birth/Death move.
    else if Perturb: **then**
        Propose a list of basins $\mathcal{B}^*$ from $q_{Perturb}(\cdot \mid \mathcal{B}^j)$, where $\mathcal{B}^*$ is $\mathcal{B}^j$ with some basin's probability distribution and probability adapted, and representative state updated. This is done by running the local sampler of some basin.
        Compute MH acceptance probability $\alpha(\mathcal{B}^* \mid \mathcal{B}^j)$ for Perturb move.
    else if Merge/Split: **then**
        Propose a list of basins $\mathcal{B}^*$ from $q_{Merge/Split}(\cdot \mid \mathcal{B}^j)$, where $\mathcal{B}^*$ is $\mathcal{B}^j$ with two merged or split basins.
        Compute MH acceptance probability $\alpha(\mathcal{B}^* \mid \mathcal{B}^j)$ for Merge/Split move.
    end if
    Accept proposal $\mathcal{B}^*$, i.e. set $\mathcal{B}^{j+1} = \mathcal{B}^*$, with probability $\alpha(\mathcal{B}^* \mid \mathcal{B}^j)$.
    Otherwise reject $\mathcal{B}^*$ and set $\mathcal{B}^{j+1} = \mathcal{B}^j$.
    Given $\alpha(\mathcal{B}^* \mid \mathcal{B}^j)$, adapt the probabilities of the basins in $\mathcal{B}^{j+1}$.
    Set $x^{j+1} = \bigcup_{B \in \mathcal{B}^{j+1}} B$.
end for

WOULD I ADAPT THE PROBABILITIES IN THIS WAY? WOULD I ADAPT THE PROPOSALS Q INSTEAD?

2.4 Development

This can become a fairly sophisticated algorithm. To develop it, I will complete the following tasks, implementing them in Python as I go:

1. **Decide initial basin probability distribution and probability:** Decide how to choose the initial probability distribution and probability for a basin given an initial representative state.
2. **Decide birth/death move:** Decide what the birth/death move should be.
3. **Decide peturb move:** Decide what the perturb moves should be. That is, decide what the local sampler for each basin should be. This includes how we propose from the basin probability distribution, and how we adapt the basin probability distribution and probability.
4. **Decide merge/split move:** Decide what the merge/split move should be.

5. **Decide move choice:** Decide how to choose a move from the set of possible moves.
6. **Prove diminishing adaption:** Prove that the adaption is diminishing, and so the chain is ergodic. Note that by construction, this MCMC is ergodic without any adaption (as long as the birth move shares support with the target density, etc).

If I have extra time, I will also complete the following tasks:

1. **Include adaption stopping mechanism:** Decide a mechanism for stopping adaption of basins once they have diminishing adaption.
2. **Include delayed acceptance step:** Decide a delayed acceptance step before computing the MH acceptance probability and doing the MH step. We would use a bogus likelihood for the delayed acceptance step, and then use the actual likelihood for the MH step. This would reduce the number of times we need to evaluate the forward map and should increase efficiency with a well chosen bogus likelihood.